

Best Methods for H₂ Molecule Simulation

For the hydrogen molecule (H₂), which is the simplest neutral molecule, we can achieve highly accurate results with relatively modest quantum resources. Here are the optimal choices for each component:

1. Molecular Specification

- **Best Geometry**: Bond length of 0.74 Å (equilibrium distance)
- **Charge**: 0 (neutral)
- **Spin Multiplicity**: 1 (singlet state)
- **Why**: This represents the ground state of H₂, which is well-characterized classically as a benchmark system.

2. Basis Set

- **Best Choice**: cc-pVDZ or even cc-pVTZ
- **Why**: Unlike larger molecules, H₂ can afford higher-quality basis sets without excessive qubit requirements. Even using cc-pVTZ is practical, giving near-chemical accuracy. With a minimal basis (STO-3G), you get only 2 spatial orbitals, while cc-pVDZ gives a more complete description with minimal additional cost.

3. Active Space

- **Best Choice**: Full orbital space (no reduction needed)
- **Why**: With H₂, even in a cc-pVDZ basis, the total system is small enough that active space reduction is unnecessary. You can include all orbitals without computational penalties.

4. Qubit Mapping

- **Best Choice**: Parity mapping with two-qubit reduction
- **Why**: This reduces the required qubits from 4 to just 2 qubits for H₂ in minimal basis, exploiting both Z₂ symmetry and particle number conservation. This is the most efficient representation possible and enables running on the smallest quantum systems.

5. Ansatz

- **Best Choice**: UCCSD or hardware-efficient ansatz with 2-3 layers
- **Why**:

- UCCSD is complete for H_2 in minimal basis (contains all possible excitations)
- With just 1-2 parameters, it can reach chemical accuracy
- Hardware-efficient ansätze also work exceptionally well due to the small Hilbert space
- This makes H_2 ideal for testing ansatz designs with minimal parameters

6. Optimizer

- **Best Choice**: COBYLA or L-BFGS-B with ~100 iterations
- **Why**:
 - The potential energy surface for H_2 is well-behaved and smooth
 - With few parameters, optimization converges quickly
 - COBYLA is gradient-free and robust for this small-scale problem
 - Even simple gradient descent works well for H_2

7. Execution Backend

- **Best Choice**: Real quantum hardware or statevector simulator
- **Why**:
 - H_2 with 2-qubit reduction creates circuits shallow enough for NISQ hardware
 - Serves as an excellent validation case for hardware performance
 - Easy to verify results against classical calculations
 - Ideal testbed for error mitigation techniques

Special Advantage of H_2

H_2 is special because it allows us to sweep through different bond distances to create a full potential energy surface with minimal computational cost. This enables:

- Calculating dissociation energies
- Finding equilibrium bond lengths
- Testing optimizer performance across different molecular configurations
- Benchmarking hardware against exact solutions

For practical implementation, I recommend using the parity mapping with Z_2 symmetry reduction to create a 2-qubit problem, then using a simple hardware-efficient ansatz with 2-3 layers. This

approach achieves chemical accuracy while being executable on the smallest quantum devices available.

Best Methods for Neon (Ne) Atom Simulation

Simulating neon is significantly more challenging than H₂ due to its 10 electrons and closed-shell electronic structure. Here are the optimal choices for each component:

1. System Specification

- **Atom**: Neon (Ne)
- **Charge**: 0 (neutral)
- **Spin Multiplicity**: 1 (closed shell, singlet)
- **Why**: Neutral ground state is most relevant for benchmarking, with all electrons paired

2. Basis Set

- **Best Choice**: cc-pVDZ or 6-31G
- **Fallback**: aug-cc-pVDZ if diffuse functions are important
- **Why**:
 - Neon requires at least a double-zeta basis for reasonable core/valence orbital description
 - STO-3G is inadequate for electron correlation in noble gases
 - Augmented basis sets help if studying polarizability or excited states

3. Active Space

- **Best Choice**: CAS(8,5) - 8 electrons in 5 orbitals
- **Why**:
 - Focus on valence electrons (2s, 2p) while freezing the 1s core electrons
 - This reduces the computational cost significantly while capturing the relevant physics

- Full active space would require 10 electrons in 10+ orbitals, which is prohibitively expensive

4. Qubit Mapping

- **Best Choice**: Bravyi-Kitaev mapping
- **Why**:
 - For systems with more electrons, BK mapping offers better locality properties
 - More efficient than Jordan-Wigner for larger atoms
 - With appropriate symmetry reductions, can save several qubits

5. Ansatz

- **Best Choice**: k-UpCCGSD (k=2 or 3)
- **Alternatives**: UCCSD with active space or hardware-efficient ansatz
- **Why**:
 - Full UCCSD would be too parameter-heavy for Ne
 - k-UpCCGSD provides a more compact circuit with fewer parameters
 - k=2 or k=3 layers are typically sufficient for reaching chemical accuracy
 - Hardware-efficient ansatz could be used for NISQ devices but may require more iterations

6. Optimizer

- **Best Choice**: L-BFGS-B or SPSA
- **Why**:
 - L-BFGS-B works well for the larger parameter space of Ne
 - More iterations required (500-1000) compared to H₂
 - SPSA is a good alternative for noisy settings or hardware runs

7. Execution Backend

- **Best Choice**: GPU-accelerated simulator
- **Why**:
 - Neon requires significantly more qubits (10+ even with reductions)
 - GPU acceleration essential for reasonable runtime
 - Real hardware execution challenging due to circuit depth and qubit count

Implementation Code Sketch

```
```python name=neon_simulation.py
import numpy as np
import matplotlib.pyplot as plt
from qiskit_nature.second_q.drivers import PySCFDriver
from qiskit_nature.second_q.mappers import BravyiKitaevMapper
from qiskit_nature.second_q.circuit.library import UCCSD, HartreeFock
from qiskit_nature.second_q.transformers import ActiveSpaceTransformer
from qiskit.algorithms import VQE
from qiskit.algorithms.optimizers import L_BFGS_B
from qiskit_aer.primitives import Estimator as AerEstimator

Neon atom specification
atom = "Ne 0.0 0.0 0.0"
basis = "cc-pVDZ"
charge = 0
spin = 0

Create driver and solve
driver = PySCFDriver(atom=atom, basis=basis, charge=charge, spin=spin)
problem = driver.run()

Reference energy calculation (for comparison)
from pyscf import gto, scf, mcscf
mol = gto.M(atom=atom, basis=basis)
mf = scf.RHF(mol).run()
print(f"HF energy: {mf.e_tot:.8f}")

Extract information about orbitals/electrons
```

```

print(f"Total molecular orbitals: {problem.num_spatial_orbitals}")
print(f"Total electrons: {sum(problem.num_particles)}")

Create active space - focus on valence electrons (8e, 5o)
Freeze 1s core electrons (first orbital)
active_space_transformer = ActiveSpaceTransformer(
 num_electrons=(4, 4), # 8 electrons (4 alpha, 4 beta)
 num_spatial_orbitals=5, # 5 spatial orbitals (2s, 2p)
 active_orbitals=[1, 2, 3, 4, 5] # Skip 1s orbital (index 0)
)

Apply the transformation
active_problem = active_space_transformer.transform(problem)
print(f"Active space: {sum(active_problem.num_particles)} electrons in
{active_problem.num_spatial_orbitals} orbitals")

Create Qubit Mapper
mapper = BravyiKitaevMapper()

Map to qubit operator
qubit_op = mapper.map(active_problem.hamiltonian.second_q_op())
print(f"Qubit operator size: {qubit_op.num_qubits} qubits")

Create ansatz (k-UpCCGSD or UCCSD with active space)
Option 1: UCCSD
init_state = HartreeFock(
 active_problem.num_spatial_orbitals,
 active_problem.num_particles,
 mapper
)

ansatz = UCCSD(

```

```

 active_problem.num_spatial_orbitals,
 active_problem.num_particles,
 mapper,
 initial_state=init_state
)

Option 2: For k-UpCCGSD, you would need a custom implementation
from custom_library import kUpCCGSD
ansatz = kUpCCGSD(
active_problem.num_spatial_orbitals,
active_problem.num_particles,
k=2, # 2 layers
mapper=mapper,
initial_state=init_state
)

print(f"Number of parameters: {ansatz.num_parameters}")

Use GPU-accelerated estimator for performance
estimator = AerEstimator(
 run_options={"shots": 8192},
 transpile_options={"optimization_level": 3},
 approximation=True,
 device="GPU"
)

Create optimizer with appropriate settings for larger system
optimizer = L_BFGS_B(maxiter=1000, iprint=10)

Prepare VQE
initial_point = 0.1 * np.random.random(ansatz.num_parameters)

```

```

vqe = VQE(estimator, ansatz, optimizer, initial_point=initial_point)

Run VQE
result = vqe.compute_minimum_eigenvalue(qubit_op)

Interpret result
electronic_energy = result.eigenvalue.real
nuclear_repulsion = 0.0 # Atom has no nuclear repulsion
total_energy = electronic_energy + nuclear_repulsion + active_problem.energy_shift

print("\n=== Neon Ground State Energy ===")
print(f"VQE Electronic Energy: {electronic_energy:.8f} Hartree")
print(f"Nuclear Repulsion Energy: {nuclear_repulsion:.8f} Hartree")
print(f"Energy Shift (from frozen core): {active_problem.energy_shift:.8f} Hartree")
print(f"Total Energy: {total_energy:.8f} Hartree")

Calculate and print ionization energy (if needed)
This would require additional calculations for Ne+ cation
...

```

## ## Key Challenges for Neon Simulation

1. **Active Space Selection**: Critical to choose the right orbitals
2. **Parameter Count**: Much larger than H<sub>2</sub>, requiring efficient optimization
3. **Qubit Requirements**: Even with reductions, requires 10+ qubits
4. **Reference Values**: Should compare against high-level classical methods like CCSD(T)

For absolute best accuracy on neon, you might consider implementing additional techniques like perturbative corrections on top of your VQE results to account for the dynamic correlation effects not captured in the reduced active space.



---

## # Best Methods for H<sub>2</sub>O (Water) Molecule Simulation

Water makes an excellent test case for quantum simulation as it's more complex than H<sub>2</sub> but more tractable than neon, with interesting molecular properties. Here are the optimal choices for each component:

### ## 1. Molecular Specification

- **Geometry**: Bent structure with O-H bond length  $\sim 0.96\text{\AA}$ , H-O-H angle  $\sim 104.5^\circ$ 
  - Example coordinates: `"O 0.0 0.0 0.0; H 0.0 0.757 0.587; H 0.0 -0.757 0.587"`
- **Charge**: 0 (neutral)
- **Spin Multiplicity**: 1 (singlet, all electrons paired)
- **Why**: This represents the ground state equilibrium geometry of water, which is well-characterized experimentally

### ## 2. Basis Set

- **Best Choice**: cc-pVDZ
- **Alternatives**: 6-31G (faster) or aug-cc-pVDZ (if studying hydrogen bonding)
- **Why**:
  - cc-pVDZ provides a good balance between accuracy and computational cost
  - Captures polarization effects important in water
  - Minimal basis sets (STO-3G) don't describe lone pairs on oxygen well

### ## 3. Active Space

- **Best Choice**: CAS(8,7) - 8 electrons in 7 orbitals
- **Why**:
  - Freezes the oxygen 1s core electrons (2 electrons)
  - Includes all valence orbitals: O(2s), O(2p), and bonding/antibonding combinations
  - This active space captures both the important  $\sigma$ -bonding and lone pairs on oxygen

### ## 4. Qubit Mapping

- **Best Choice**: Parity mapping with two-qubit reduction
- **Why**:
  - Exploits particle number conservation and point group symmetry
  - Reduces qubit count significantly (from 14 to 12 qubits with active space)
  - More efficient than Jordan-Wigner for systems with symmetry

### ## 5. Ansatz

- **Best Choice**: k-UpCCGSD (k=2)
- **Alternatives**:
  - UCCSD with active space (more accurate but higher circuit depth)
  - Hardware-efficient ansatz with custom entanglement (more NISQ-friendly)
- **Why**:
  - k-UpCCGSD provides a good balance of expressivity and circuit depth
  - Two layers (k=2) typically sufficient for water's electronic structure
  - More parameter-efficient than full UCCSD

### ## 6. Optimizer

- **Best Choice**: SLSQP or L-BFGS-B
- **Parameters**: 300-500 iterations
- **Why**:
  - SLSQP works well for water's smooth energy landscape
  - L-BFGS-B effective for the moderate parameter count
  - Water requires more iterations than H<sub>2</sub> but fewer than neon

### ## 7. Execution Backend

- **Best Choice**: GPU-accelerated simulator
- **Alternatives**:
  - Statevector simulator (for reference calculations)
  - Real quantum hardware only with significant active space reduction
- **Why**:
  - With active space reduction, water simulation benefits from GPU acceleration

- Still too large for reliable execution on current NISQ hardware without error mitigation

## ## Implementation Code

```
```python name=water_simulation.py
import numpy as np
import matplotlib.pyplot as plt
from qiskit_nature.second_q.drivers import PySCFDriver
from qiskit_nature.units import DistanceUnit
from qiskit_nature.second_q.mappers import ParityMapper
from qiskit_nature.second_q.circuit.library import HartreeFock, UCCSD
from qiskit_nature.second_q.transformers import ActiveSpaceTransformer
from qiskit_algorithms import VQE
from qiskit_algorithms.optimizers import SLSQP
from qiskit_aer.primitives import Estimator as AerEstimator
from pyscf import gto, scf, mcscf

# Water molecule specification (equilibrium geometry)
ATOM_STR = "O 0.0 0.0 0.0; H 0.0 0.757 0.587; H 0.0 -0.757 0.587"
BASIS = "cc-pVDZ" # Good balance for water

# --- PySCF Molecule ---
def get_pyscf_mol():
    mol = gto.Mole()
    mol.atom = ATOM_STR
    mol.basis = BASIS
    mol.charge = 0
    mol.spin = 0
    mol.build()
    return mol
```

```
# --- Qiskit Nature Problem ---
```

```
def get_qiskit_problem():
```

```
    driver = PySCFDriver(
```

```
        atom=ATOM_STR,
```

```
        unit=DistanceUnit.ANGSTROM,
```

```
        charge=0,
```

```
        spin=0,
```

```
        basis=BASIS
```

```
    )
```

```
    return driver.run()
```

```
# --- VQE with GPU Acceleration ---
```

```
def run_vqe(problem):
```

```
    # 1. Apply Active Space Transformation - focus on valence electrons
```

```
    active_space_transformer = ActiveSpaceTransformer(
```

```
        num_electrons=(4, 4), # 8 valence electrons (4 alpha, 4 beta)
```

```
        num_spatial_orbitals=7, # 7 spatial orbitals
```

```
        # Skip the oxygen 1s orbital (index 0)
```

```
        active_orbitals=[1, 2, 3, 4, 5, 6, 7]
```

```
    )
```

```
    active_problem = active_space_transformer.transform(problem)
```

```
    print(f"Active space: {sum(active_problem.num_particles)} electrons in  
{active_problem.num_spatial_orbitals} orbitals")
```

```
    # 2. Use Parity mapping with Z2 symmetry reduction
```

```
    mapper = ParityMapper(num_particles=active_problem.num_particles)
```

```
    # 3. Create ansatz with appropriate initial state
```

```
    init_state = HartreeFock(active_problem.num_spatial_orbitals, active_problem.num_particles,  
                             mapper)
```

```
    ansatz = UCCSD(active_problem.num_spatial_orbitals, active_problem.num_particles, mapper,  
                   initial_state=init_state)
```

```

print(f"UCCSD parameters: {ansatz.num_parameters}")

# 4. Better initial point (small random values with seed for reproducibility)
np.random.seed(42)
initial_point = 0.1 * np.random.random(ansatz.num_parameters)

# 5. Use GPU-accelerated estimator
try:
    estimator = AerEstimator(
        run_options={"shots": 8192},
        approximation=True,
        device="GPU"
    )
    print("Using GPU acceleration")
except Exception:
    print("GPU not available, falling back to CPU")
    estimator = AerEstimator(run_options={"shots": 8192})

# 6. SLSQP optimizer with appropriate iterations
optimizer = SLSQP(maxiter=400)

# 7. Run VQE
vqe = VQE(estimator, ansatz, optimizer, initial_point=initial_point)
qubit_op = mapper.map(active_problem.hamiltonian.second_q_op())
print(f"Qubit operator size: {qubit_op.num_qubits} qubits")

result = vqe.compute_minimum_eigenvalue(qubit_op)

# 8. Return interpreted energy including the energy shift from frozen core
return active_problem.interpret(result).total_energies[0].real

```

```

# --- Reference CASSCF calculation ---

def run_casscf(mol):
    # Run mean-field first
    myhf = scf.RHF(mol)
    myhf.kernel()

    # Run CASSCF with same active space as VQE (8e, 7o)
    mycas = mcscf.CASSCF(myhf, 7, 8)
    casscf_energy = mycas.kernel()[0]
    return casscf_energy, myhf.e_tot

# --- Calculate dipole moment ---

def calculate_dipole(mol):
    myhf = scf.RHF(mol)
    myhf.kernel()
    return myhf.dip_moment()

# --- Main ---

try:
    # Quantum calculation
    problem = get_qiskit_problem()
    print(f"Basis: {BASIS}")
    print(f"Full system - Spatial orbitals: {problem.num_spatial_orbitals}, Particles: {problem.num_particles}")

    vqe_energy = run_vqe(problem)

    # Classical reference calculations
    mol = get_pyscf_mol()
    casscf_energy, hf_energy = run_casscf(mol)
    dipole = calculate_dipole(mol)

```

```

# Energy results
print("\n=== Water (H2O) Ground State Properties ===")
print(f"VQE Energy: {vqe_energy:.6f} Hartree")
print(f"HF Energy: {hf_energy:.6f} Hartree")
print(f"CASSCF(8e,7o) Energy: {casscf_energy:.6f} Hartree")
print(f"Correlation energy: {vqe_energy - hf_energy:.6f} Hartree")

# Dipole moment
print(f"\nDipole moment: {np.linalg.norm(dipole):.4f} Debye")
print(f"Dipole components (x,y,z): {dipole}")

except Exception as e:
    print("Calculation failed:", str(e))
    import traceback
    traceback.print_exc()
'''

## Unique Considerations for H2O

1. Symmetry Exploitation: Water's  $C_{2v}$  symmetry can reduce computational cost.

2. Dipole Moment: Unlike H2 and Ne, water has a significant dipole moment (1.85 Debye experimentally) that can be calculated as an observable.

3. Multiple Properties: Water is ideal for calculating both energetic and non-energetic properties (dipole moments, vibrational frequencies).

4. Hydrogen Bonding: If studying water clusters, consider augmented basis sets to capture hydrogen bonding effects.

```

5. **Frozen Natural Orbitals**: For more advanced studies, using frozen natural orbitals can provide better active spaces than canonical orbitals.

Water represents an ideal "middle ground" test case - more complex than diatomics but more tractable than larger molecules, making it an excellent benchmark system for quantum algorithms.

Best Practices for CH₄ (Methane) Molecule Simulation

Methane presents an interesting quantum simulation target - more complex than H₂ or H₂O, but still manageable with proper active space selection. Here's a specialized implementation for CH₄ using best practices:

```
```python name=methane_simulation.py
import numpy as np
import matplotlib.pyplot as plt
import time
from datetime import datetime

from qiskit_nature.second_q.drivers import PySCFDriver
from qiskit_nature.units import DistanceUnit
from qiskit_nature.second_q.mappers import ParityMapper
from qiskit_nature.second_q.circuit.library import UCCSD, HartreeFock
from qiskit_nature.second_q.transformers import ActiveSpaceTransformer
from qiskit.primitives import Estimator
from qiskit_algorithms import VQE
from qiskit_algorithms.optimizers import SLSQP

from pyscf import gto, scf, mcscf
```



```
class MethaneSim:
```

```
 """Specialized class for methane (CH4) quantum simulation."""
```

```
 def __init__(self, verbose=True):
```

```
 """Initialize methane simulation with optimal settings."""
```

```
 self.verbose = verbose
```

```
 self.results = {}
```

```
 # Best configuration for methane
```

```
 self.basis = "6-31G" # Good balance for CH4
```

```
 self.charge = 0
```

```
 self.spin = 0
```

```
 self.active_space = {
```

```
 "num_electrons": (4, 4), # 8 valence electrons
```

```
 "num_spatial_orbitals": 8, # 8 spatial orbitals
```

```
 "active_orbitals": [1, 2, 3, 4, 5, 6, 7, 8] # Skip C 1s core orbital
```

```
 }
```

```
 self.mapper_type = "parity"
```

```
 self.optimizer_name = "SLSQP"
```

```
 self.optimizer_kwargs = {"maxiter": 500}
```

```
 # Print configuration
```

```
 if self.verbose:
```

```
 self._print_config()
```

```
 def _print_config(self):
```

```
 """Print configuration details."""
```

```
 print(f"\n{'=' * 50}")
```

```
 print(f"Configuration for CH4 (Methane)")
```

```
 print(f"{'=' * 50}")
```

```
 print(f"Basis set: {self.basis}")
```

```

print(f"Charge: {self.charge}, Spin: {self.spin}")
print(f"Active space: {self.active_space}")
print(f"Mapper: {self.mapper_type}")
print(f"Optimizer: {self.optimizer_name} with {self.optimizer_kwargs}")
print(f'{'=' * 50}\n')

```

```

def get_methane_geometry(self, scale_factor=1.0):

```

```

 """

```

```

 Generate tetrahedral methane geometry.

```

```

 Args:

```

```

 scale_factor: Scale the C-H bond length (1.0 = equilibrium ~1.09 Å)

```

```

 Returns:

```

```

 List of tuples with atom type and coordinates

```

```

 """

```

```

 # Equilibrium C-H bond length in methane is ~1.09 Å

```

```

 bond_length = 1.09 * scale_factor

```

```

 # Tetrahedral geometry

```

```

 a = bond_length / np.sqrt(3)

```

```

 return [

```

```

 ("C", (0.0, 0.0, 0.0)),

```

```

 ("H", (a, a, a)),

```

```

 ("H", (-a, -a, a)),

```

```

 ("H", (-a, a, -a)),

```

```

 ("H", (a, -a, -a))

```

```

]

```

```

def _format_geometry(self, geometry):

```

```

 """Format geometry into string specification."""
 atom_spec = ""
 for atom in geometry:
 atom_spec += f"{atom[0]} {atom[1][0]} {atom[1][1]} {atom[1][2]}; "
 return atom_spec.strip("; ")

 def _get_pyscf_mol(self, geometry):
 """Create a PySCF molecule object."""
 atom_spec = self._format_geometry(geometry)
 mol = gto.Mole()
 mol.atom = atom_spec
 mol.basis = self.basis
 mol.charge = self.charge
 mol.spin = self.spin
 mol.build()
 return mol

 def _get_qiskit_problem(self, geometry):
 """Create a Qiskit problem object."""
 atom_spec = self._format_geometry(geometry)
 driver = PySCFDriver(
 atom=atom_spec,
 unit=DistanceUnit.ANGSTROM,
 charge=self.charge,
 spin=self.spin,
 basis=self.basis
)
 problem = driver.run()

 # Apply active space transformation
 transformer = ActiveSpaceTransformer(

```

```

 num_electrons=self.active_space["num_electrons"],
 num_spatial_orbitals=self.active_space["num_spatial_orbitals"],
 active_orbitals=self.active_space["active_orbitals"]
)
 problem = transformer.transform(problem)

 if self.verbose:
 print(f'Applied active space: {sum(problem.num_particles)} electrons in
 {problem.num_spatial_orbitals} orbitals")

 return problem

def run_vqe(self, problem):
 """Run VQE on the given problem."""

 # Create mapper (Parity with qubit reduction)
 mapper = ParityMapper(num_particles=problem.num_particles)

 # Create initial state and ansatz
 init_state = HartreeFock(
 problem.num_spatial_orbitals,
 problem.num_particles,
 mapper
)
 ansatz = UCCSD(
 problem.num_spatial_orbitals,
 problem.num_particles,
 mapper,
 initial_state=init_state
)

 # Use random initial parameters for better convergence

```

```

np.random.seed(42) # For reproducibility
initial_point = 0.1 * np.random.random(ansatz.num_parameters)

Create optimizer and estimator
optimizer = SLSQP(**self.optimizer_kwargs)
estimator = Estimator() # CPU-based estimator

Create and run VQE
vqe = VQE(estimator, ansatz, optimizer, initial_point=initial_point)
qubit_op = mapper.map(problem.hamiltonian.second_q_op())

if self.verbose:
 print(f"Ansatz parameters: {ansatz.num_parameters}")
 print(f"Qubit operator size: {qubit_op.num_qubits} qubits")

start_time = time.time()
result = vqe.compute_minimum_eigenvalue(qubit_op)
runtime = time.time() - start_time

if self.verbose:
 print(f"VQE runtime: {runtime:.2f} seconds")

Interpret and return energy
return problem.interpret(result).total_energies[0].real

def run_reference_calcs(self, mol):
 """Run reference classical calculations."""
 results = {}

 # Run HF
 try:

```

```

mf = scf.RHF(mol)
results["hf"] = mf.kernel()
except Exception as e:
 print(f"HF calculation failed: {e}")
 results["hf"] = None

Run CASSCF
try:
 mf_cas = scf.RHF(mol).run()
 mycas = mcscf.CASSCF(
 mf_cas,
 self.active_space["num_spatial_orbitals"],
 sum(self.active_space["num_electrons"])
)
 results["casscf"] = mycas.kernel()[0]
except Exception as e:
 print(f"CASSCF calculation failed: {e}")
 results["casscf"] = None

FCI on methane is likely too expensive, but we could try MP2
try:
 mf_mp2 = scf.RHF(mol).run()
 mp2_energy = mol.MP2().kernel()[0]
 results["mp2"] = mp2_energy + mf_mp2.e_tot
except Exception as e:
 print(f"MP2 calculation failed: {e}")
 results["mp2"] = None

return results

def run_bond_scan(self, start=0.8, end=1.2, points=9):

```

```
"""
```

Run energy scan over C-H bond length scaling.

Args:

start: Starting scale factor (0.8 = 80% of equilibrium)

end: Ending scale factor (1.2 = 120% of equilibrium)

points: Number of points in the scan

```
"""
```

```
scale_factors = np.linspace(start, end, points)
```

```
distances = scale_factors * 1.09 # Convert scale factors to actual C-H distances
```

```
vqe_energies, hf_energies, casscf_energies, mp2_energies = [], [], [], []
```

```
for i, scale in enumerate(scale_factors):
```

```
 if self.verbose:
```

```
 print(f"\nPoint {i+1}/{len(scale_factors)} at C-H bond scaling {scale:.2f} (bond length: {distances[i]:.3f} Å)")
```

```
 # Get geometry and create molecule
```

```
 geometry = self.get_methane_geometry(scale)
```

```
 mol = self._get_pyscf_mol(geometry)
```

```
 # Run reference calculations
```

```
 ref_results = self.run_reference_calcs(mol)
```

```
 hf_energies.append(ref_results.get("hf"))
```

```
 casscf_energies.append(ref_results.get("casscf"))
```

```
 mp2_energies.append(ref_results.get("mp2"))
```

```
 # Run VQE calculation
```

```
 try:
```

```
 problem = self._get_qiskit_problem(geometry)
```

```

 vqe_energy = self.run_vqe(problem)
 vqe_energies.append(vqe_energy)

 # Print energies

 print(f" Scale: {scale:.2f} | C-H dist: {distances[i]:.3f} Å | VQE: {vqe_energy:.6f} | HF:
{ref_results.get('hf', 'N/A'):.6f}")

 if ref_results.get("casscf") is not None:
 print(f" CASSCF: {ref_results.get('casscf'):.6f}")

 if ref_results.get("mp2") is not None:
 print(f" MP2: {ref_results.get('mp2'):.6f}")

except Exception as e:
 print(f"VQE calculation failed at scale {scale:.2f}: {e}")
 vqe_energies.append(np.nan)

Store results
self.results["distances"] = distances
self.results["vqe"] = vqe_energies
self.results["hf"] = hf_energies
self.results["casscf"] = casscf_energies
self.results["mp2"] = mp2_energies

return self.results

def run_equilibrium(self):
 """Run single point calculation at equilibrium geometry."""
 print("\nRunning single point calculation at equilibrium geometry...")

 # Get geometry and create molecule
 geometry = self.get_methane_geometry(1.0)
 mol = self._get_pyscf_mol(geometry)

```



```

Run reference calculations
ref_results = self.run_reference_calcs(mol)

Run VQE calculation
try:
 problem = self._get_qiskit_problem(geometry)
 vqe_energy = self.run_vqe(problem)

Store results
self.results["equilibrium"] = {
 "vqe": vqe_energy,
 "hf": ref_results.get("hf"),
 "casscf": ref_results.get("casscf"),
 "mp2": ref_results.get("mp2")
}

print(f"\nEquilibrium CH4 energies:")
print(f" VQE: {vqe_energy:.6f} Hartree")
print(f" HF: {ref_results.get('hf', 'N/A'):.6f} Hartree")
if ref_results.get("casscf") is not None:
 print(f" CASSCF: {ref_results.get('casscf'):.6f} Hartree")
if ref_results.get("mp2") is not None:
 print(f" MP2: {ref_results.get('mp2'):.6f} Hartree")

return self.results["equilibrium"]

except Exception as e:
 print(f"VQE calculation failed: {e}")
 return None

```

```

def plot_results(self, save_path=None):
 """Plot energy scan results."""
 if "distances" not in self.results:
 print("No bond scan results to plot. Run bond_scan first.")
 return

 plt.figure(figsize=(8, 6))

 # Plot VQE energies
 plt.plot(
 self.results["distances"],
 self.results["vqe"],
 'o-',
 label='VQE',
 color='limegreen',
 linewidth=2
)

 # Plot HF energies
 if any(e is not None for e in self.results["hf"]):
 plt.plot(
 self.results["distances"],
 self.results["hf"],
 's-',
 label='HF',
 color='red',
 linewidth=2
)

 # Plot CASSCF energies
 if any(e is not None for e in self.results["casscf"]):

```

```

plt.plot(
 self.results["distances"],
 self.results["casscf"],
 '*',
 label='CASSCF',
 color='blue',
 linewidth=2
)

Plot MP2 energies
if any(e is not None for e in self.results["mp2"]):
 plt.plot(
 self.results["distances"],
 self.results["mp2"],
 'x-',
 label='MP2',
 color='purple',
 linewidth=2
)

Add plot details
plt.xlabel("C-H Bond Length (Å)", fontsize=12)
plt.ylabel("Energy (Hartree)", fontsize=12)
plt.title(f"CH4 Potential Energy Curve ({self.basis} basis)", fontsize=14)
plt.grid(True, alpha=0.3)
plt.legend(fontsize=10)
plt.tight_layout()

Save or show
if save_path:
 plt.savefig(save_path, dpi=300)

```

```

 print(f"Plot saved to {save_path}")
 else:
 plt.show()

def get_summary(self):
 """Get summary of results."""
 if "equilibrium" in self.results:
 # Single point summary
 eq = self.results["equilibrium"]

 summary = f"\n{'=' * 30} CH4 RESULTS SUMMARY {'=' * 30}\n"
 summary += f"Basis set: {self.basis}\n"
 summary += f"VQE energy: {eq['vqe']:.8f} Hartree\n"

 if eq["hf"] is not None:
 summary += f"HF energy: {eq['hf']:.8f} Hartree\n"
 summary += f"Correlation energy: {eq['vqe'] - eq['hf']:.8f} Hartree\n"

 if eq["casscf"] is not None:
 summary += f"CASSCF energy: {eq['casscf']:.8f} Hartree\n"
 summary += f"VQE error vs CASSCF: {(eq['vqe'] - eq['casscf']) * 1000:.3f} mHartree\n"

 if eq["mp2"] is not None:
 summary += f"MP2 energy: {eq['mp2']:.8f} Hartree\n"
 summary += f"VQE error vs MP2: {(eq['vqe'] - eq['mp2']) * 1000:.3f} mHartree\n"

 elif "distances" in self.results:
 # Bond scan summary
 min_vqe_idx = np.nanargmin(self.results["vqe"])
 min_dist = self.results["distances"][min_vqe_idx]
 min_vqe = self.results["vqe"][min_vqe_idx]

```

```

summary = f"\n{'=' * 30} CH4 RESULTS SUMMARY {'=' * 30}\n"
summary += f"Basis set: {self.basis}\n"
summary += f"Minimum VQE energy: {min_vqe:.8f} Hartree at C-H bond length {min_dist:.3f} Å\n"

if self.results["hf"][min_vqe_idx] is not None:
 hf_energy = self.results["hf"][min_vqe_idx]
 summary += f"HF energy: {hf_energy:.8f} Hartree\n"
 summary += f"Correlation energy: {min_vqe - hf_energy:.8f} Hartree\n"

if self.results["casscf"] and self.results["casscf"][min_vqe_idx] is not None:
 casscf_energy = self.results["casscf"][min_vqe_idx]
 summary += f"CASSCF energy: {casscf_energy:.8f} Hartree\n"
 summary += f"VQE error vs CASSCF: {(min_vqe - casscf_energy) * 1000:.3f} mHartree\n"

if self.results["mp2"] and self.results["mp2"][min_vqe_idx] is not None:
 mp2_energy = self.results["mp2"][min_vqe_idx]
 summary += f"MP2 energy: {mp2_energy:.8f} Hartree\n"
 summary += f"VQE error vs MP2: {(min_vqe - mp2_energy) * 1000:.3f} mHartree\n"
else:
 return "No results to summarize."

summary += f"\n{'=' * 80}\n"
return summary

```

# Example usage

```
if __name__ == "__main__":
```

```
 # Print timestamp
```

```
 print(f"Run started: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
```

```

Create methane simulator
sim = MethaneSim(verbose=True)

Option 1: Run single point at equilibrium geometry
sim.run_equilibrium()

Option 2: Run bond scan
results = sim.run_bond_scan(start=0.8, end=1.2, points=5)
sim.plot_results()

Print summary
print(sim.get_summary())
...

```

## ## Best Practices for CH<sub>4</sub> Simulation

### 1. **Molecular Structure**:

- Tetrahedral geometry with 109.5° H-C-H angles
- C-H bond length ~1.09 Å
- All C-H bonds identical (symmetry can be exploited)

### 2. **Basis Set**: `6-31G`

- Better than STO-3G for describing polarization in C-H bonds
- Balances accuracy with computational efficiency
- cc-pVDZ would be more accurate but significantly more expensive

### 3. **Active Space Selection**: CAS(8,8)

- 8 electrons in 8 orbitals
- Freezes C 1s core (2 electrons)
- Includes all valence orbitals involved in C-H bonding

- This reduction is essential for tractable simulation

4. **Qubit Mapping**:

- Parity mapping with particle number conservation
- Takes advantage of methane's symmetry

5. **Ansatz**: UCCSD with active space

- Most accurate for describing electron correlation
- With the reduced active space, parameter count is manageable

6. **Optimizer**:

- SLSQP with 500 iterations
- Efficient for methane's moderately complex parameter landscape

7. **Reference Calculations**:

- CASSCF with the same active space
- MP2 for dynamic correlation comparison
- FCI would be too expensive for methane

## ## Key Features of the Implementation

1. **Tetrahedral Geometry Generator**: Correctly builds methane's structure with variable bond lengths
2. **Bond Length Scanning**: Maps the potential energy surface for C-H bond stretching
3. **Multiple Reference Methods**: Compares against HF, CASSCF, and MP2
4. **Active Space Focus**: Concentrates computational resources on chemically relevant orbitals
5. **Random Parameter Initialization**: Improves convergence over zero initialization

To run this code, simply execute the file and it will:

- Run a single-point calculation at equilibrium geometry
- Compare VQE results with classical reference methods

- Provide a detailed summary of energies and accuracy

For further exploration, you can uncomment the bond scan section to generate a full potential energy curve.

---

## # Best Practices for N<sub>2</sub> Molecule Simulation

I'll analyze your code and suggest the optimal configurations for N<sub>2</sub>, which presents unique challenges due to its triple bond and strongly correlated electrons.

### ## Analysis of Your Current Code

#### ### Strengths:

1.  **\*\*Active Space Approach\*\***: Using a (6e,6o) active space is a reasonable starting point
2.  **\*\*Parity Mapper with particle conservation\*\***: Good choice for N<sub>2</sub>
3.  **\*\*UCCSD with HF Initial State\*\***: Appropriate ansatz choice
4.  **\*\*Bond Length Scan\*\***: Excellent for mapping the dissociation curve

#### ### Areas for Improvement:

1.  **\*\*Basis Set\*\***: `sto3g` is insufficient for N<sub>2</sub>, especially for the triple bond
2.  **\*\*Active Space Selection\*\***: Not explicitly targeting the most relevant orbitals
3.  **\*\*Optimizer Settings\*\***: COBYLA with 200 iterations may not converge for N<sub>2</sub>'s complex landscape
4.  **\*\*Parameter Initialization\*\***: Using all zeros is suboptimal

### ## Optimal Configuration for N<sub>2</sub>

Here's the improved implementation with best practices for N<sub>2</sub>:

```
```python name=n2_improved_simulation.py
```



```

import numpy as np
import matplotlib.pyplot as plt
import time

from qiskit.primitives import Estimator
from qiskit_algorithms import VQE
from qiskit_algorithms.optimizers import L_BFGS_B, SLSQP

from qiskit_nature.second_q.drivers import PySCFDriver
from qiskit_nature.units import DistanceUnit
from qiskit_nature.second_q.transformers import ActiveSpaceTransformer
from qiskit_nature.second_q.mappers import ParityMapper
from qiskit_nature.second_q.circuit.library import HartreeFock, UCCSD

# PySCF for classical references
from pyscf import gto, scf, fci, mcscf

# --- Best settings for N2 ---
BASIS = "6-31g" # Better basis for N2 (compromise between accuracy and cost)
# Active space: focus on 2p orbitals involved in triple bond + lone pairs
ACTIVE_ELECTRONS = (5, 5) # 10 valence electrons (5 alpha, 5 beta)
ACTIVE_SPATIAL_ORBITALS = 8 # Including 2s and 2p orbitals
ACTIVE_ORBITALS = [2, 3, 4, 5, 6, 7, 8, 9] # Skip 1s core orbitals (indices 0,1)
MAX_ITER = 500 # More iterations for N2's complex landscape

# Bond distances to scan (N-N bond in Angstrom)
# Include more points around equilibrium (1.098 Å)
DISTS = np.concatenate([
    np.linspace(0.9, 1.3, 9), # More points around equilibrium
    np.linspace(1.4, 2.6, 7) # Fewer points in dissociation region
])

```

```

# --- Helper functions ---

def build_pyscf_mol(distance, basis=BASIS):
    # Linear molecule along z-axis: N at -d/2 and +d/2
    atom = f"N 0 0 {-distance/2}; N 0 0 {distance/2}"
    mol = gto.Mole()
    mol.atom = atom
    mol.basis = basis
    mol.charge = 0
    mol.spin = 0
    mol.build()
    return mol

def run_references(mol):
    """Run multiple reference calculations."""
    results = {}

    # RHF
    try:
        mf = scf.RHF(mol)
        results["rhf"] = mf.kernel()
        mf_obj = mf
    except Exception as e:
        print(f"RHF failed: {e}")
        results["rhf"] = np.nan
        mf_obj = None

    # CASSCF with same active space as VQE
    try:
        mf_cas = scf.RHF(mol).run()
        mycas = mcscf.CASSCF(mf_cas,

```

```

        nelecas=sum(ACTIVE_ELECTRONS),
        ncas=ACTIVE_SPATIAL_ORBITALS)

    results["casscf"] = mycas.kernel()[0]
except Exception as e:
    print(f"CASSCF failed: {e}")
    results["casscf"] = np.nan

# Try FCI in active space if feasible
if mf_obj is not None:
    try:
        # Create a smaller molecule with just the active space
        cisolver = fci.FCI(mol, mf_obj.mo_coeff)
        e, _ = cisolver.kernel()
        results["fci"] = e
    except Exception as e:
        print(f"FCI failed (likely too large): {e}")
        results["fci"] = np.nan

return results

def build_qiskit_problem(distance, basis=BASIS):
    """Build Qiskit Nature problem with proper active space."""
    atom = f"N 0 0 {-distance/2}; N 0 0 {distance/2}"
    driver = PySCFDriver(atom=atom, basis=basis, unit=DistanceUnit.ANGSTROM,
                          charge=0, spin=0)
    problem = driver.run()

    # Print full system information
    print(f"Full system: {problem.num_spatial_orbitals} orbitals, "
          f"{problem.num_particles} electrons")

```

```

# Apply active space transformer with explicit orbital selection
transformer = ActiveSpaceTransformer(
    num_electrons=ACTIVE_ELECTRONS,
    num_spatial_orbitals=ACTIVE_SPATIAL_ORBITALS,
    active_orbitals=ACTIVE_ORBITALS # Skip core 1s orbitals
)
problem_reduced = transformer.transform(problem)

print(f"Active space: {sum(problem_reduced.num_particles)} electrons in "
      f"{problem_reduced.num_spatial_orbitals} orbitals")

return problem_reduced

def run_vqe(problem):
    """Run VQE with improved settings."""
    # Use parity mapper with symmetry preservation
    mapper = ParityMapper(num_particles=problem.num_particles)

    # Create initial state and ansatz
    init_state = HartreeFock(problem.num_spatial_orbitals,
                             problem.num_particles,
                             mapper)
    ansatz = UCCSD(problem.num_spatial_orbitals,
                   problem.num_particles,
                   mapper,
                   initial_state=init_state)

    print(f"UCCSD parameters: {ansatz.num_parameters}")

    # Better initialization - small random values
    np.random.seed(42) # For reproducibility

```

```

initial_point = 0.1 * np.random.random(ansatz.num_parameters)

# Use L-BFGS-B optimizer - better for N2's landscape
optimizer = L_BFGS_B(maxiter=MAX_ITER)

# Create CPU estimator
estimator = Estimator()

# Create and run VQE
vqe = VQE(estimator, ansatz, optimizer, initial_point=initial_point)
qubit_op = mapper.map(problem.hamiltonian.second_q_op())
print(f"Qubit operator size: {qubit_op.num_qubits} qubits")

# Run VQE and time it
start_time = time.time()
result = vqe.compute_minimum_eigenvalue(qubit_op)
runtime = time.time() - start_time

# Get interpreted energy
energy = problem.interpret(result).total_energies[0].real

print(f"VQE completed in {runtime:.1f} seconds")
print(f"VQE energy: {energy:.8f} Hartree")
print(f"Optimizer converged: {result.optimizer_evals} iterations")

return energy, result, qubit_op

# --- Main sweep ---
print(f"\n{' '*50}")
print(f"N2 Potential Energy Surface with VQE")
print(f"Basis: {BASIS}")

```

```
print(f"Active space: {sum(ACTIVE_ELECTRONS)}e, {ACTIVE_SPATIAL_ORBITALS}o")
print(f"{' '*50}")
```

```
vqe_energies = []
rhf_energies = []
casscf_energies = []
fci_energies = []
```

```
for d in DISTS:
```

```
    print(f"\n{' '*30}")
    print(f"Distance: {d:.3f} Å")
    print(f"{' '*30}")
```

```
# Run classical references
```

```
try:
```

```
    mol = build_pyscf_mol(d)
    ref_results = run_references(mol)
    rhf_energies.append(ref_results.get("rhf", np.nan))
    casscf_energies.append(ref_results.get("casscf", np.nan))
    fci_energies.append(ref_results.get("fci", np.nan))
```

```
except Exception as e:
```

```
    print(f"Reference calculations failed: {e}")
    rhf_energies.append(np.nan)
    casscf_energies.append(np.nan)
    fci_energies.append(np.nan)
```

```
# Run VQE
```

```
try:
```

```
    problem = build_qiskit_problem(d)
    vqe_e, _, _ = run_vqe(problem)
    vqe_energies.append(vqe_e)
```

```

except Exception as e:
    print(f"VQE calculation failed: {e}")
    vqe_energies.append(np.nan)

# --- Calculate N2 dissociation energy ---
try:
    # Find minimum energy
    min_idx = np.nanargmin(vqe_energies)
    min_energy = vqe_energies[min_idx]
    eq_distance = DISTS[min_idx]

    # Energy at large separation (approximates 2 x N atom)
    dissoc_idx = -1 # Last point (largest separation)
    dissoc_energy = vqe_energies[dissoc_idx]

    # Dissociation energy
    de = min_energy - dissoc_energy
    print(f"\nDissociation energy: {-de:.6f} Hartree ({-de*627.5:.2f} kcal/mol)")
except Exception as e:
    print(f"Could not calculate dissociation energy: {e}")

# --- Plotting ---
plt.figure(figsize=(10, 6))

# Plot VQE energies
plt.plot(DISTS, vqe_energies, 'o-', label='VQE (active space)',
         color='limegreen', linewidth=2, markersize=8)

# Plot RHF energies
if any(~np.isnan(rhf_energies)):
    plt.plot(DISTS, rhf_energies, 's--', label='RHF',

```

```

        color='red', linewidth=2, markersize=6)

# Plot CASSCF energies
if any(~np.isnan(casscf_energies)):
    plt.plot(DISTS, casscf_energies, 'd-', label='CASSCF',
             color='blue', linewidth=2, markersize=6)

# Plot FCI energies if available
if any(~np.isnan(fci_energies)):
    plt.plot(DISTS, fci_energies, 'x--', label='FCI',
             color='black', linewidth=2, markersize=6)

# Plot styling
plt.xlabel("N–N Distance (Å)", fontsize=14)
plt.ylabel("Energy (Hartree)", fontsize=14)
plt.title(f"N2 Potential Energy Curve ({BASIS} basis)", fontsize=16)
plt.legend(fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()

# Add text box with summary
try:
    textstr = f"Equilibrium distance: {eq_distance:.3f} Å\n"
    textstr += f"VQE minimum energy: {min_energy:.6f} Ha\n"
    textstr += f"Dissociation energy: {-de*627.5:.1f} kcal/mol"

    props = dict(boxstyle='round', facecolor='wheat', alpha=0.4)
    plt.text(0.05, 0.05, textstr, transform=plt.gca().transAxes,
            fontsize=12, verticalalignment='bottom', bbox=props)
except:
    pass

```



```

plt.savefig('n2_potential_curve.png', dpi=300)

plt.show()

# --- Report minimum from VQE sweep ---
if np.all(np.isnan(vqe_energies)):
    print("\nNo successful VQE energies calculated.")
else:
    idx = int(np.nanargmin(vqe_energies))
    print("\n=== Minimum VQE result (from sweep) ===")
    print(f"Equilibrium distance: {DISTS[idx]:.3f} Å")
    print(f"VQE energy: {vqe_energies[idx]:.8f} Hartree")

# Compare with references
if not np.isnan(rhf_energies[idx]):
    print(f"RHF energy: {rhf_energies[idx]:.8f} Hartree")
    print(f"Correlation energy: {vqe_energies[idx] - rhf_energies[idx]:.8f} Hartree")

if not np.isnan(casscf_energies[idx]):
    print(f"CASSCF energy: {casscf_energies[idx]:.8f} Hartree")
    print(f"VQE vs CASSCF: {(vqe_energies[idx] - casscf_energies[idx])*1000:.3f} mHartree")
...

```

Best Practices for N₂ Molecule

1. **Basis Set**: `6-31g` is recommended (better than sto3g)

- Provides improved description of triple bond
- Balances accuracy with computational cost
- cc-pVDZ would be ideal but very expensive for N₂

2. **Active Space**: (10e,8o) focused on valence

- 10 valence electrons (5 alpha, 5 beta)
 - 8 spatial orbitals (2s and 2p from each N)
 - Explicitly skip the 1s core orbitals
 - This captures the $\text{N}\equiv\text{N}$ triple bond and lone pairs
3. **Qubit Mapping**: Parity mapper with symmetry
- Preserves particle number conservation
 - Reduces qubit count
 - Better suited for strongly correlated systems like N_2
4. **Ansatz**: UCCSD with HF initial state
- Essential for capturing strong correlation in triple bond
 - Parameter count is manageable with active space
5. **Optimizer**: L-BFGS-B with 500 iterations
- More robust than COBYLA for N_2 's complex energy landscape
 - Handles the steep potential well at equilibrium
6. **Parameter Initialization**:
- Small random values (0.1 scale)
 - Avoids getting stuck in local minima
 - Improves convergence rate
7. **Reference Calculations**:
- CASSCF with matching active space
 - Best classical reference for comparison
8. **Distance Sampling**:
- Dense sampling near equilibrium ($\sim 1.1\text{\AA}$)
 - Captures triple bond breaking during dissociation

The key challenge with N_2 is the triple bond, which requires a larger active space and more computational resources than molecules like H_2 or H_2O . This optimized implementation provides the best balance of accuracy and efficiency for CPU-based simulation of N_2 .

Best Practices for C_2H_2 (Acetylene) Simulation

Looking at your code for acetylene simulation, I can recommend several optimizations for this challenging molecule with its triple bond.

Analysis of Current Implementation

Strengths:

1.  ****Correct Geometry****: Linear arrangement with proper bond lengths
2.  ****Active Space Approach****: Reducing to 8e in 8o is reasonable
3.  ****Parity Mapper****: Good choice with particle number conservation
4.  ****CC-Distance Scan****: Appropriate range covering equilibrium ($\sim 1.2\text{\AA}$)

Areas for Improvement:

1.  ****Basis Set****: "sto3g" is insufficient for triple bond character
2.  ****Active Space Selection****: No explicit orbital targeting
3.  ****Optimizer****: COBYLA with 200 iterations is insufficient for C_2H_2 's complex landscape
4.  ****Reference Methods****: Only UHF, which doesn't capture static correlation
5.  ****Parameter Initialization****: Using all zeros is suboptimal

Optimal Implementation for C_2H_2

```
```python name=c2h2_improved_simulation.py
import numpy as np
import matplotlib.pyplot as plt
```

```

import time

from datetime import datetime

from qiskit_nature.second_q.drivers import PySCFDriver
from qiskit_nature.units import DistanceUnit
from qiskit_nature.second_q.transformers import ActiveSpaceTransformer
from qiskit_nature.second_q.mappers import ParityMapper
from qiskit_nature.second_q.circuit.library import HartreeFock, UCCSD

from qiskit.primitives import Estimator
from qiskit_algorithms import VQE
from qiskit_algorithms.optimizers import SLSQP, L_BFGS_B

from pyscf import gto, scf, mcscf, mp

--- Best settings for C2H2 ---
BASIS = "6-31g" # Better basis for acetylene
Active space: focus on triple bond and C-H bonds
ACTIVE_ELECTRONS = (5, 5) # 10 valence electrons (5 alpha, 5 beta)
ACTIVE_SPATIAL_ORBITALS = 10 # 10 spatial orbitals for valence space
Skip C 1s core orbitals (first 2 orbitals for each C)
ACTIVE_ORBITALS = list(range(4, 14)) # Skip 4 core orbitals
MAX_ITER = 400 # More iterations for C2H2's complex landscape

class AcetyleneSimulator:
 """Class for simulating acetylene (C2H2) using VQE with best practices."""

 def __init__(self, verbose=True):
 self.verbose = verbose
 self.results = {}

```

if self.verbose:

```
print(f'\n{'='*50}')
```

```
print("Acetylene (C2H2) VQE Simulation with Best Practices")
```

```
print(f{'='*50}')
```

```
print(f"Basis: {BASIS}")
```

```
print(f"Active Space: {sum(ACTIVE_ELECTRONS)}e in {ACTIVE_SPATIAL_ORBITALS}o")
```

```
print(f{'='*50}\n")
```

```
def get_c2h2_geometry(self, cc_distance=1.2, ch_distance=1.06):
```

```
 """
```

Generate linear acetylene geometry.

Args:

cc\_distance: C≡C bond length in Å (default: 1.2)

ch\_distance: C-H bond length in Å (default: 1.06)

Returns:

List of tuples with atom type and coordinates

```
 """
```

```
 return [
```

```
 ("C", (0.0, 0.0, -cc_distance/2)),
```

```
 ("C", (0.0, 0.0, cc_distance/2)),
```

```
 ("H", (0.0, 0.0, -cc_distance/2 - ch_distance)),
```

```
 ("H", (0.0, 0.0, cc_distance/2 + ch_distance))
```

```
]
```

```
def format_geometry(self, geometry):
```

```
 """Convert geometry to string format."""
```

```
 atom_str = ""
```

```
 for atom in geometry:
```

```
 atom_str += f"{atom[0]} {atom[1][0]} {atom[1][1]} {atom[1][2]}; "
```

```
return atom_str.strip("; ")
```

```
def build_pyscf_mol(self, geometry):
```

```
 """Create PySCF molecule object."""
```

```
 atom_str = self.format_geometry(geometry)
```

```
 mol = gto.Mole()
```

```
 mol.atom = atom_str
```

```
 mol.basis = BASIS
```

```
 mol.charge = 0
```

```
 mol.spin = 0
```

```
 mol.build()
```

```
 return mol
```

```
def run_reference_calcs(self, mol):
```

```
 """Run reference classical calculations."""
```

```
 results = {}
```

```
 # Run RHF
```

```
 try:
```

```
 mf = scf.RHF(mol)
```

```
 results["rhf"] = mf.kernel()
```

```
 mf_obj = mf
```

```
 except Exception as e:
```

```
 print(f"RHF calculation failed: {e}")
```

```
 results["rhf"] = np.nan
```

```
 mf_obj = None
```

```
 # Run MP2 for dynamic correlation
```

```
 try:
```

```
 mp2 = mp.MP2(mf_obj)
```

```
 mp2.kernel()
```

```

 # MP2 correlation energy + RHF energy
 results["mp2"] = mp2.e_corr + results["rhf"]
except Exception as e:
 print(f"MP2 calculation failed: {e}")
 results["mp2"] = np.nan

```

```

Run CASSCF with same active space as VQE
try:
 mf_cas = scf.RHF(mol).run()
 mycas = mcscf.CASSCF(mf_cas,
 ncas=ACTIVE_SPATIAL_ORBITALS,
 nelecas=sum(ACTIVE_ELECTRONS))
 results["casscf"] = mycas.kernel()[0]
except Exception as e:
 print(f"CASSCF calculation failed: {e}")
 results["casscf"] = np.nan

```

```

return results

```

```

def build_qiskit_problem(self, geometry):
 """Build Qiskit Nature problem with proper active space."""
 atom_str = self.format_geometry(geometry)

```

```

Create driver and run
driver = PySCFDriver(
 atom=atom_str,
 basis=BASIS,
 unit=DistanceUnit.ANGSTROM,
 charge=0,
 spin=0
)

```

```

problem = driver.run()

Print full system information
if self.verbose:
 print(f"Full system: {problem.num_spatial_orbitals} orbitals, "
 f"{problem.num_particles} electrons")

Apply active space transformation with explicit orbital selection
transformer = ActiveSpaceTransformer(
 num_electrons=ACTIVE_ELECTRONS,
 num_spatial_orbitals=ACTIVE_SPATIAL_ORBITALS,
 active_orbitals=ACTIVE_ORBITALS # Skip core orbitals
)
problem = transformer.transform(problem)

if self.verbose:
 print(f"Active space: {sum(problem.num_particles)} electrons in "
 f"{problem.num_spatial_orbitals} orbitals")

return problem

def run_vqe(self, problem):
 """Run VQE with optimal settings."""
 # Use parity mapper with symmetry reduction
 mapper = ParityMapper(num_particles=problem.num_particles)

 # Create initial state and ansatz
 init_state = HartreeFock(
 problem.num_spatial_orbitals,
 problem.num_particles,
 mapper

```



```
)
```

```
ansatz = UCCSD(
 problem.num_spatial_orbitals,
 problem.num_particles,
 mapper,
 initial_state=init_state
)
```

```
if self.verbose:
```

```
 print(f"UCCSD parameters: {ansatz.num_parameters}")
```

```
Better initialization - small random values with seed for reproducibility
```

```
np.random.seed(42)
```

```
initial_point = 0.1 * np.random.random(ansatz.num_parameters)
```

```
Use SLSQP optimizer - works well for C2H2
```

```
optimizer = SLSQP(maxiter=MAX_ITER)
```

```
Create CPU estimator
```

```
estimator = Estimator()
```

```
Create and run VQE
```

```
vqe = VQE(estimator, ansatz, optimizer, initial_point=initial_point)
```

```
qubit_op = mapper.map(problem.hamiltonian.second_q_op())
```

```
if self.verbose:
```

```
 print(f"Qubit operator size: {qubit_op.num_qubits} qubits")
```

```
Run VQE and time it
```

```
start_time = time.time()
```

```

result = vqe.compute_minimum_eigenvalue(qubit_op)
runtime = time.time() - start_time

Get interpreted energy
energy = problem.interpret(result).total_energies[0].real

if self.verbose:
 print(f"VQE completed in {runtime:.1f} seconds")
 print(f"VQE energy: {energy:.8f} Hartree")
 print(f"Optimizer evaluations: {result.optimizer_evals}")

return energy, result

def run_cc_scan(self, cc_distances=None, ch_distance=1.06):
 """
 Run energy scan over C≡C bond distances.

 Args:
 cc_distances: List of C≡C distances to scan (default: around equilibrium)
 ch_distance: Fixed C-H bond length (default: 1.06 Å)
 """
 if cc_distances is None:
 # Default distances: focused around equilibrium (~1.2 Å)
 cc_distances = np.linspace(1.0, 1.6, 9)

 vqe_energies = []
 rhf_energies = []
 mp2_energies = []
 casscf_energies = []

 for i, cc_d in enumerate(cc_distances):

```

```

print(f"\n{'-'*30}")
print(f"C≡C Distance: {cc_d:.3f} Å ({i+1}/{len(cc_distances)})")
print(f"{'-'*30}")

Generate geometry
geometry = self.get_c2h2_geometry(cc_distance=cc_d, ch_distance=ch_distance)

Run classical references
mol = self.build_pyscf_mol(geometry)
ref_results = self.run_reference_calcs(mol)
rhf_energies.append(ref_results.get("rhf"))
mp2_energies.append(ref_results.get("mp2"))
casscf_energies.append(ref_results.get("casscf"))

Run VQE
try:
 problem = self.build_qiskit_problem(geometry)
 vqe_energy, _ = self.run_vqe(problem)
 vqe_energies.append(vqe_energy)

Print comparison
print("\nEnergy comparison at CC distance = {:.3f} Å:".format(cc_d))
print(f" VQE: {vqe_energy:.8f} Hartree")
print(f" RHF: {ref_results.get('rhf', 'N/A')}")
if ref_results.get("mp2") is not None:
 print(f" MP2: {ref_results.get('mp2'):.8f} Hartree")
if ref_results.get("casscf") is not None:
 print(f" CASSCF: {ref_results.get('casscf'):.8f} Hartree")

except Exception as e:
 print(f"VQE calculation failed: {e}")

```

```

 vqe_energies.append(np.nan)

Store results
self.results = {
 'cc_distances': cc_distances,
 'vqe': vqe_energies,
 'rhf': rhf_energies,
 'mp2': mp2_energies,
 'casscf': casscf_energies
}

return self.results

def run_ch_scan(self, cc_distance=1.2, ch_distances=None):
 """
 Run energy scan over C-H bond distances with fixed C≡C distance.

 Args:
 cc_distance: Fixed C≡C bond length (default: 1.2 Å)
 ch_distances: List of C-H distances to scan
 """
 if ch_distances is None:
 # Default: scan around equilibrium C-H distance
 ch_distances = np.linspace(0.9, 1.3, 7)

 vqe_energies = []
 rhf_energies = []
 casscf_energies = []

 for i, ch_d in enumerate(ch_distances):
 print(f"\n{'-'*30}")

```

```

print(f"C-H Distance: {ch_d:.3f} Å ({i+1}/{len(ch_distances)})")

print(f"{'-'*30}")

Generate geometry
geometry = self.get_c2h2_geometry(cc_distance=cc_distance, ch_distance=ch_d)

Run classical references and VQE
(Implementation similar to run_cc_scan)
...

Store and return results
...

def plot_results(self, save_path=None, scan_type='cc'):
 """Plot energy scan results."""
 if not self.results:
 print("No results to plot. Run a scan first.")
 return

 plt.figure(figsize=(10, 6))

 if scan_type == 'cc':
 distances = self.results['cc_distances']
 x_label = "C≡C Distance (Å)"
 title = f"C2H2 C≡C Bond Scan ({BASIS} basis)"
 else:
 distances = self.results['ch_distances']
 x_label = "C-H Distance (Å)"
 title = f"C2H2 C-H Bond Scan ({BASIS} basis)"

 # Plot VQE energies

```

```
plt.plot(
 distances,
 self.results['vqe'],
 'o-',
 label='VQE (active space)',
 color='limegreen',
 linewidth=2,
 markersize=8
)
```

# Plot RHF energies

if any(~np.isnan(self.results['rhf'])):

```
plt.plot(
 distances,
 self.results['rhf'],
 's--',
 label='RHF',
 color='red',
 linewidth=2,
 markersize=6
)
```

# Plot MP2 energies if available

if 'mp2' in self.results and any(~np.isnan(self.results['mp2'])):

```
plt.plot(
 distances,
 self.results['mp2'],
 'x-',
 label='MP2',
 color='purple',
 linewidth=2,
```

```

 markersize=6
)

Plot CASSCF energies
if any(~np.isnan(self.results['casscf'])):
 plt.plot(
 distances,
 self.results['casscf'],
 'd-',
 label='CASSCF',
 color='blue',
 linewidth=2,
 markersize=6
)

Plot styling
plt.xlabel(x_label, fontsize=14)
plt.ylabel("Energy (Hartree)", fontsize=14)
plt.title(title, fontsize=16)
plt.legend(fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()

Add text box with equilibrium information
try:
 vqe_min_idx = np.nanargmin(self.results['vqe'])
 eq_distance = distances[vqe_min_idx]
 min_energy = self.results['vqe'][vqe_min_idx]

 textstr = f"Equilibrium distance: {eq_distance:.3f} Å\n"
 textstr += f"VQE minimum energy: {min_energy:.6f} Ha"

```

```

 props = dict(boxstyle='round', facecolor='wheat', alpha=0.4)
 plt.text(0.05, 0.05, textstr, transform=plt.gca().transAxes,
 fontsize=12, verticalalignment='bottom', bbox=props)
 except:
 pass

 if save_path:
 plt.savefig(save_path, dpi=300)
 print(f"Plot saved to {save_path}")
 else:
 plt.show()

def get_summary(self):
 """Get summary of results."""
 if not self.results:
 return "No results to summarize."

 try:
 # Find minimum energies
 vqe_min_idx = np.nanargmin(self.results['vqe'])

 if 'cc_distances' in self.results:
 distances = self.results['cc_distances']
 distance_type = "C≡C"
 else:
 distances = self.results['ch_distances']
 distance_type = "C-H"

 eq_distance = distances[vqe_min_idx]
 min_vqe = self.results['vqe'][vqe_min_idx]

```



```

summary = f"\n{'=' * 30} C2H2 RESULTS SUMMARY {'=' * 30}\n"
summary += f"Basis set: {BASIS}\n"
summary += f"Active space: {sum(ACTIVE_ELECTRONS)}e in {ACTIVE_SPATIAL_ORBITALS}o\n"
summary += f"Equilibrium {distance_type} distance: {eq_distance:.3f} Å\n"
summary += f"VQE minimum energy: {min_vqe:.8f} Hartree\n"

```

```

Compare with reference values

```

```

if not np.isnan(self.results['rhf'][vqe_min_idx]):
 rhf_energy = self.results['rhf'][vqe_min_idx]
 summary += f"RHF energy: {rhf_energy:.8f} Hartree\n"
 summary += f"Correlation energy: {min_vqe - rhf_energy:.8f} Hartree\n"

```

```

if 'mp2' in self.results and not np.isnan(self.results['mp2'][vqe_min_idx]):
 mp2_energy = self.results['mp2'][vqe_min_idx]
 summary += f"MP2 energy: {mp2_energy:.8f} Hartree\n"
 summary += f"VQE error vs MP2: {(min_vqe - mp2_energy) * 1000:.3f} mHartree\n"

```

```

if not np.isnan(self.results['casscf'][vqe_min_idx]):
 casscf_energy = self.results['casscf'][vqe_min_idx]
 summary += f"CASSCF energy: {casscf_energy:.8f} Hartree\n"
 summary += f"VQE error vs CASSCF: {(min_vqe - casscf_energy) * 1000:.3f} mHartree\n"

```

```

summary += f"\n{'=' * 80}\n"

```

```

return summary

```

```

except Exception as e:

```

```

 return f"Error generating summary: {e}"

```

```

Main execution

```

```

if __name__ == "__main__":

```

```

print(f"C2H2 simulation started: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")

Create simulator
sim = AcetyleneSimulator(verbose=True)

Run C≡C bond scan
results = sim.run_cc_scan()

Plot results
sim.plot_results(save_path="c2h2_potential_curve.png")

Print summary
print(sim.get_summary())

Optionally run C-H bond scan at equilibrium C≡C distance
min_cc_idx = np.nanargmin(sim.results['vqe'])
eq_cc_distance = sim.results['cc_distances'][min_cc_idx]
ch_results = sim.run_ch_scan(cc_distance=eq_cc_distance)
...

```

### ## Best Practices for C<sub>2</sub>H<sub>2</sub> (Acetylene) Simulation

#### 1. **Molecular Structure**:

- Linear geometry with 180° bond angles
- C≡C bond length ~1.2Å (triple bond)
- C-H bond length ~1.06Å
- Total of 14 electrons (8 from carbons, 2 from hydrogens)

#### 2. **Basis Set**: `6-31g`

- Much better than sto3g for triple bond description
- Balances accuracy with computational cost

- cc-pVDZ would be ideal but significantly more expensive

3. **Active Space Selection**: CAS(10e,10o)

- 10 valence electrons in 10 spatial orbitals
- Freeze C 1s core orbitals (4 electrons)
- Includes all bonding/antibonding  $\pi$  orbitals essential for triple bond
- Explicitly selecting orbitals 4-13 (skipping 0-3 core orbitals)

4. **Qubit Mapping**:

- Parity mapping with particle number conservation
- Exploits symmetries to reduce qubit count

5. **Ansatz**:

- UCCSD with active space (best accuracy)
- HF initial state preparation

6. **Optimizer**:

- SLSQP with 400+ iterations
- Small random initial parameters (0.1 scale)
- More robust convergence for acetylene's complex landscape

7. **Reference Calculations**:

- RHF: Basic reference
- MP2: Captures dynamic correlation
- CASSCF: Best reference for strongly correlated triple bond

8. **Enhanced Analyses**:

- Both  $\text{C}\equiv\text{C}$  and C-H bond scans
- Correlation energy calculation
- Proper comparison with classical methods

The key challenge with  $C_2H_2$  is the triple bond between the carbon atoms, which requires careful active space selection to capture the strong correlation effects. This implementation provides the optimal balance between accuracy and computational efficiency for simulating acetylene on a CPU-based quantum simulator.

---

#### # Best Practices for $C_2H_4$ (Ethylene) Simulation

Ethylene presents unique computational challenges due to its  $C=C$  double bond with  $\pi$ -bonding, making it an excellent test molecule for quantum algorithms. Here's an optimized implementation with best practices:

```
```python name=c2h4_simulation.py

import numpy as np
import matplotlib.pyplot as plt
import time
from datetime import datetime

from qiskit_nature.second_q.drivers import PySCFDriver
from qiskit_nature.units import DistanceUnit
from qiskit_nature.second_q.transformers import ActiveSpaceTransformer
from qiskit_nature.second_q.mappers import ParityMapper
from qiskit_nature.second_q.circuit.library import HartreeFock, UCCSD

from qiskit.primitives import Estimator
from qiskit_algorithms import VQE
from qiskit_algorithms.optimizers import SLSQP, L_BFGS_B

from pyscf import gto, scf, mcscf, mp

# --- Best settings for  $C_2H_4$  ---
```

```

BASIS = "6-31g" # Good compromise for ethylene
# Active space: focus on double bond and C-H bonds
ACTIVE_ELECTRONS = (6, 6) # 12 valence electrons (6 alpha, 6 beta)
ACTIVE_SPATIAL_ORBITALS = 12 # 12 spatial orbitals for valence space
# Skip C 1s core orbitals (first 2 orbitals for each C)
ACTIVE_ORBITALS = list(range(4, 16)) # Skip 4 core orbitals
MAX_ITER = 400 # Good number of iterations for ethylene

```

class EthyleneSimulator:

```

    """Class for simulating ethylene (C2H4) using VQE with best practices."""

```

```

    def __init__(self, verbose=True):

```

```

        self.verbose = verbose

```

```

        self.results = {}

```

```

        if self.verbose:

```

```

            print(f"\n{' '*50}")

```

```

            print("Ethylene (C2H4) VQE Simulation with Best Practices")

```

```

            print(f"{' '*50}")

```

```

            print(f"Basis: {BASIS}")

```

```

            print(f"Active Space: {sum(ACTIVE_ELECTRONS)}e in {ACTIVE_SPATIAL_ORBITALS}o")

```

```

            print(f"{' '*50}\n")

```

```

    def get_c2h4_geometry(self, cc_distance=1.33):

```

```

        """

```

```

        Generate planar ethylene geometry.

```

```

        Args:

```

```

            cc_distance: C=C bond length in Å (default: 1.33)

```

```

        Returns:

```

List of tuples with atom type and coordinates

"""

Standard bond parameters

ch_distance = 1.09 # Å

h_angle = 120.0 # degrees

Convert to radians

h_angle_rad = h_angle * np.pi / 180.0

Calculate coordinates

x_h = ch_distance * np.sin(h_angle_rad)

y_h = ch_distance * np.cos(h_angle_rad)

return [

Carbon atoms along z-axis

("C", (0.0, 0.0, -cc_distance/2)),

("C", (0.0, 0.0, cc_distance/2)),

Hydrogen atoms in the xz plane

("H", (x_h, 0.0, -cc_distance/2)),

("H", (-x_h, 0.0, -cc_distance/2)),

("H", (x_h, 0.0, cc_distance/2)),

("H", (-x_h, 0.0, cc_distance/2)),

]

def format_geometry(self, geometry):

"""Convert geometry to string format."""

atom_str = ""

for atom in geometry:

atom_str += f"{atom[0]} {atom[1][0]} {atom[1][1]} {atom[1][2]}; "

return atom_str.strip("; ")

```

def build_pyscf_mol(self, geometry):
    """Create PySCF molecule object."""
    atom_str = self.format_geometry(geometry)
    mol = gto.Mole()
    mol.atom = atom_str
    mol.basis = BASIS
    mol.charge = 0
    mol.spin = 0
    mol.build()
    return mol

def run_reference_calcs(self, mol):
    """Run reference classical calculations."""
    results = {}

    # Run RHF
    try:
        mf = scf.RHF(mol)
        results["rhf"] = mf.kernel()
        mf_obj = mf
    except Exception as e:
        print(f"RHF calculation failed: {e}")
        results["rhf"] = np.nan
        mf_obj = None

    # Run MP2 for dynamic correlation
    try:
        mp2 = mp.MP2(mf_obj)
        mp2.kernel()

        # MP2 correlation energy + RHF energy
        results["mp2"] = mp2.e_corr + results["rhf"]

```

```

except Exception as e:
    print(f"MP2 calculation failed: {e}")
    results["mp2"] = np.nan

# Run CASSCF with same active space as VQE
try:
    mf_cas = scf.RHF(mol).run()
    mycas = mcscf.CASSCF(mf_cas,
                        ncas=ACTIVE_SPATIAL_ORBITALS,
                        nelecas=sum(ACTIVE_ELECTRONS))
    results["casscf"] = mycas.kernel()[0]
except Exception as e:
    print(f"CASSCF calculation failed: {e}")
    results["casscf"] = np.nan

return results

```

```

def build_qiskit_problem(self, geometry):
    """Build Qiskit Nature problem with proper active space."""
    atom_str = self.format_geometry(geometry)

    # Create driver and run
    driver = PySCFDriver(
        atom=atom_str,
        basis=BASIS,
        unit=DistanceUnit.ANGSTROM,
        charge=0,
        spin=0
    )
    problem = driver.run()

```



```

# Print full system information

if self.verbose:

    print(f"Full system: {problem.num_spatial_orbitals} orbitals, "
          f"{problem.num_particles} electrons")


# Apply active space transformation with explicit orbital selection
transformer = ActiveSpaceTransformer(
    num_electrons=ACTIVE_ELECTRONS,
    num_spatial_orbitals=ACTIVE_SPATIAL_ORBITALS,
    active_orbitals=ACTIVE_ORBITALS # Skip core orbitals
)
problem = transformer.transform(problem)


if self.verbose:

    print(f"Active space: {sum(problem.num_particles)} electrons in "
          f"{problem.num_spatial_orbitals} orbitals")


return problem


def run_vqe(self, problem):
    """Run VQE with optimal settings."""

    # Use parity mapper with symmetry reduction
    mapper = ParityMapper(num_particles=problem.num_particles)

    # Create initial state and ansatz
    init_state = HartreeFock(
        problem.num_spatial_orbitals,
        problem.num_particles,
        mapper
    )

```

```

ansatz = UCCSD(
    problem.num_spatial_orbitals,
    problem.num_particles,
    mapper,
    initial_state=init_state
)

if self.verbose:
    print(f"UCCSD parameters: {ansatz.num_parameters}")

# Better initialization - small random values with seed for reproducibility
np.random.seed(42)
initial_point = 0.1 * np.random.random(ansatz.num_parameters)

# Use SLSQP optimizer - works well for ethylene
optimizer = SLSQP(maxiter=MAX_ITER)

# Create CPU estimator
estimator = Estimator()

# Create and run VQE
vqe = VQE(estimator, ansatz, optimizer, initial_point=initial_point)
qubit_op = mapper.map(problem.hamiltonian.second_q_op())

if self.verbose:
    print(f"Qubit operator size: {qubit_op.num_qubits} qubits")

# Run VQE and time it
start_time = time.time()

result = vqe.compute_minimum_eigenvalue(qubit_op)

runtime = time.time() - start_time

```

```
# Get interpreted energy
```

```
energy = problem.interpret(result).total_energies[0].real
```

```
if self.verbose:
```

```
    print(f"VQE completed in {runtime:.1f} seconds")
```

```
    print(f"VQE energy: {energy:.8f} Hartree")
```

```
    print(f"Optimizer evaluations: {result.optimizer_evals}")
```

```
return energy, result
```

```
def run_cc_scan(self, cc_distances=None):
```

```
    """
```

```
    Run energy scan over C=C bond distances.
```

```
    Args:
```

```
        cc_distances: List of C=C distances to scan (default: around equilibrium)
```

```
    """
```

```
    if cc_distances is None:
```

```
        # Default distances: focused around equilibrium (~1.33 Å)
```

```
        cc_distances = np.linspace(1.1, 1.6, 9)
```

```
    vqe_energies = []
```

```
    rhf_energies = []
```

```
    mp2_energies = []
```

```
    casscf_energies = []
```

```
    for i, cc_d in enumerate(cc_distances):
```

```
        print(f"\n{'-'*30}")
```

```
        print(f"C=C Distance: {cc_d:.3f} Å ({i+1}/{len(cc_distances)})")
```

```
        print(f"{'-'*30}")
```

```

# Generate geometry
geometry = self.get_c2h4_geometry(cc_distance=cc_d)

# Run classical references
mol = self.build_pyscf_mol(geometry)
ref_results = self.run_reference_calcs(mol)
rhf_energies.append(ref_results.get("rhf"))
mp2_energies.append(ref_results.get("mp2"))
casscf_energies.append(ref_results.get("casscf"))

# Run VQE
try:
    problem = self.build_qiskit_problem(geometry)
    vqe_energy, _ = self.run_vqe(problem)
    vqe_energies.append(vqe_energy)

# Print comparison
print("\nEnergy comparison at C=C distance = {:.3f} Å:".format(cc_d))
print(f" VQE:  {vqe_energy:.8f} Hartree")
print(f" RHF:  {ref_results.get('rhf', 'N/A')}")
if ref_results.get("mp2") is not None:
    print(f" MP2:  {ref_results.get('mp2'):.8f} Hartree")
if ref_results.get("casscf") is not None:
    print(f" CASSCF: {ref_results.get('casscf'):.8f} Hartree")

except Exception as e:
    print(f"VQE calculation failed: {e}")
    vqe_energies.append(np.nan)

# Store results

```

```
self.results = {  
    'cc_distances': cc_distances,  
    'vqe': vqe_energies,  
    'rhf': rhf_energies,  
    'mp2': mp2_energies,  
    'casscf': casscf_energies  
}
```

```
return self.results
```

```
def plot_results(self, save_path=None):  
    """Plot energy scan results."""  
    if not self.results:  
        print("No results to plot. Run a scan first.")  
        return
```

```
plt.figure(figsize=(10, 6))
```

```
distances = self.results['cc_distances']
```

```
# Plot VQE energies
```

```
plt.plot(  
    distances,  
    self.results['vqe'],  
    'o-',  
    label='VQE (active space)',  
    color='limegreen',  
    linewidth=2,  
    markersize=8  
)
```

```
# Plot RHF energies
```

```
if any(~np.isnan(self.results['rhf'])):
```

```
    plt.plot(
        distances,
        self.results['rhf'],
        's--',
        label='RHF',
        color='red',
        linewidth=2,
        markersize=6
    )
```

```
# Plot MP2 energies if available
```

```
if 'mp2' in self.results and any(~np.isnan(self.results['mp2'])):
```

```
    plt.plot(
        distances,
        self.results['mp2'],
        'x-',
        label='MP2',
        color='purple',
        linewidth=2,
        markersize=6
    )
```

```
# Plot CASSCF energies
```

```
if any(~np.isnan(self.results['casscf'])):
```

```
    plt.plot(
        distances,
        self.results['casscf'],
        'd-',
        label='CASSCF',
```

```

        color='blue',
        linewidth=2,
        markersize=6
    )

# Plot styling
plt.xlabel("C=C Distance (Å)", fontsize=14)
plt.ylabel("Energy (Hartree)", fontsize=14)
plt.title(f"C2H4 Potential Energy Curve ({BASIS} basis)", fontsize=16)
plt.legend(fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()

# Add text box with equilibrium information
try:
    vqe_min_idx = np.nanargmin(self.results['vqe'])
    eq_distance = distances[vqe_min_idx]
    min_energy = self.results['vqe'][vqe_min_idx]

    textstr = f"Equilibrium C=C distance: {eq_distance:.3f} Å\n"
    textstr += f"VQE minimum energy: {min_energy:.6f} Ha"

    props = dict(boxstyle='round', facecolor='wheat', alpha=0.4)
    plt.text(0.05, 0.05, textstr, transform=plt.gca().transAxes,
            fontsize=12, verticalalignment='bottom', bbox=props)
except:
    pass

if save_path:
    plt.savefig(save_path, dpi=300)
    print(f"Plot saved to {save_path}")

```

else:

plt.show()

def get_summary(self):

"""Get summary of results."""

if not self.results:

return "No results to summarize."

try:

Find minimum energies

vqe_min_idx = np.nanargmin(self.results['vqe'])

distances = self.results['cc_distances']

eq_distance = distances[vqe_min_idx]

min_vqe = self.results['vqe'][vqe_min_idx]

summary = f"\n{'=' * 30} C₂H₄ RESULTS SUMMARY {'=' * 30}\n"

summary += f"Basis set: {BASIS}\n"

summary += f"Active space: {sum(ACTIVE_ELECTRONS)}e in {ACTIVE_SPATIAL_ORBITALS}o\n"

summary += f"Equilibrium C=C distance: {eq_distance:.3f} Å (exp. ~1.33 Å)\n"

summary += f"VQE minimum energy: {min_vqe:.8f} Hartree\n"

Compare with reference values

if not np.isnan(self.results['rhf'][vqe_min_idx]):

rhf_energy = self.results['rhf'][vqe_min_idx]

summary += f"RHF energy: {rhf_energy:.8f} Hartree\n"

summary += f"Correlation energy: {min_vqe - rhf_energy:.8f} Hartree\n"

if 'mp2' in self.results and not np.isnan(self.results['mp2'][vqe_min_idx]):

mp2_energy = self.results['mp2'][vqe_min_idx]

summary += f"MP2 energy: {mp2_energy:.8f} Hartree\n"

summary += f"VQE error vs MP2: {(min_vqe - mp2_energy) * 1000:.3f} mHartree\n"


```

        if not np.isnan(self.results['casscf'][vqe_min_idx]):
            casscf_energy = self.results['casscf'][vqe_min_idx]
            summary += f"CASSCF energy: {casscf_energy:.8f} Hartree\n"
            summary += f"VQE error vs CASSCF: {(min_vqe - casscf_energy) * 1000:.3f} mHartree\n"

        summary += f"'=' * 80}\n"

    return summary

except Exception as e:
    return f"Error generating summary: {e}"

# Main execution
if __name__ == "__main__":
    print(f"C2H4 simulation started: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")

    # Create simulator
    sim = EthyleneSimulator(verbose=True)

    # Run C=C bond scan
    results = sim.run_cc_scan()

    # Plot results
    sim.plot_results(save_path="c2h4_potential_curve.png")

    # Print summary
    print(sim.get_summary())
    ...

## Best Practices for C2H4 (Ethylene) Simulation

```

1. **Molecular Structure**:

- Planar geometry with sp^2 hybridized carbons
- C=C bond length $\sim 1.33\text{\AA}$ (double bond)
- C-H bond length $\sim 1.09\text{\AA}$
- H-C-H angle $\sim 120^\circ$

2. **Basis Set**: `6-31g`

- Provides good description of C=C π -bonding orbital
- Better than minimal basis sets for capturing double bond character
- Balanced computational cost for ethylene

3. **Active Space**: CAS(12e,12o)

- 12 valence electrons in 12 spatial orbitals
- Freeze C 1s core orbitals (4 electrons)
- Includes σ and π bonding/antibonding orbitals
- Selection via orbital indices 4-15 (skipping core orbitals)

4. **Qubit Mapping**: Parity with symmetry reduction

- Exploits electron number conservation
- Reduces qubit count significantly
- Better than Jordan-Wigner for this system

5. **Ansatz**: UCCSD with active space

- Captures both static and dynamic correlation
- Well-suited for π -bond in ethylene
- Parameters initialized to small random values

6. **Optimizer**: SLSQP with 400 iterations

- More efficient than COBYLA for ethylene
- Better convergence properties for the PES

7. **Reference Methods**:

- RHF: Basic reference
- MP2: Accounts for dynamic correlation
- CASSCF: Captures static correlation in C=C π -system

8. **Geometry Analysis**:

- C=C bond scan around equilibrium (1.33Å)
- Includes correlation energy analysis
- Comparison with classical reference methods

Ethylene is particularly well-suited for quantum simulation as it features a π -bond with interesting electronic properties while remaining computationally manageable with the appropriate active space. This implementation captures the essential quantum chemistry of the C=C double bond while keeping the computational requirements reasonable for CPU-based quantum simulation.